[1부] 머신러닝
9장 비지도 학습

핸즈온 머신러닝(3판)

ㅇㅇ대학교 ㅇㅇ학과
홍길동

시작하기 전에

지은이: 오렐리앙 제롱 Aurélien Géron

머신러닝 컨설턴트. 2013년에서 2016년까지 구글에서 유튜브 동영상 분류 팀을 이끌었다. 2002년에서 2012년까지 프랑스의 모바일 ISP 선두주자인 위퍼스트(Wifirst)를 설립하고 CTO로 일했다. 2001년에는 폴리콘셀(Polyconseil)을 설립하고 CTO로 일했다. 이 회사는 지금 전기차 공유 서비스인 오토립(Autolib')을 운영하고 있다.
그전에는 재무(J. P. 모건과 소시에테 제네랄(Société Générale)), 방위(캐나다 국방부), 의료(수혈) 등 다양한 분야에서 엔지니어로 일했고, C++, WiFi, 인터넷 구조에 관한 몇 권의 기술 서적을 썼으며 한 프랑스 공과대학에서 컴퓨터과학을 가르쳤다.

옮긴이: 박해선 haesun.park@tensorflow.blog

기계공학을 전공했지만 졸업 후엔 줄곧 코드를 읽고 쓰는 일을 했다. 블로그(*tensorflow.blog*)에 글을 쓰고 머신러닝과 딥러닝에 관한 책을 집필, 번역하면서 소프트웨어와 과학의 경계를 흥미롭게 탐험하고 있다.
『챗GPT로 대화하는 기술』(한빛미디어, 2023), 『혼자 공부하는 머신러닝+딥러닝』(한빛미디어, 2020), 『혼자 공부하는 데이터 분석 with 파이썬』(한빛미디어, 2023), 『Do it! 딥러닝 입문』(이지스퍼블리싱, 2019) 등 집필했다.

[코드 예제] *https://bit.ly/homl3-git* 에서 주피터 노트북으로 제공

3판의 주요 변경 내용

1. 최신 라이브러리 버전으로 전체 코드 업데이트
2. 특성 이름 추적, 히스토그램 기반 그레이디언트 부스팅, 레이블 전파 등 사이킷런에 새롭게 추가된 다양한 기능
3. 하이퍼파라미터 튜닝을 위한 케라스 튜너(Keras Tuner) 라이브러리, 자연어 처리를 위한 허깅 페이스(Hugging Face)의 트랜스포머스(Transformers) 라이브러리 및 케라스의 새로운 전처리 및 데이터 증식 층
4. 여러 비전 모델(ResNeXt, DenseNet, MobileNet, CSPNet, EfficientNet)과 올바른 모델을 선택하기 위한 가이드라인
5. <15장 RNN과 CNN을 사용한 시퀀스 처리>: 합성된 시계열 대신 시카고 버스 및 철도 탑승객 데이터를 분석하며 ARMA 모델과 그 변형
6. <16장 RNN과 어텐션을 사용한 자연어 처리>: 인코더-디코더 RNN, 트랜스포머 모델을 사용한 영어-스페인어 번역 모델 구축, 스위치 트랜스포머(Switch Transformer), DistilBERT, T5, PaLM(사고 사슬 프롬프트 포함)과 같은 언어 모델 비전 트랜스포머(ViT) 소개, DeiT, 퍼시비어, DINO와 같은 트랜스포머 기반 비전 모델을 비롯한 CLIP, DALL·E, 플라밍고, GATO 등 몇 가지 대형 멀티모달 모델 개요
7. <17장 오토인코더, GAN 그리고 확산 모델>: 확산 모델 소개, DDPM 구현
8. <19장 대규모 텐서플로 모델 훈련과 배포>: 구글 클라우드 AI 플랫폼에서 구글 버텍스 AI로 마이그레이션, 대규모 하이퍼파라미터 검색을 위한 분산 케라스 튜너, TensorFlow.js 코드
9. PipeDream과 Pathways를 비롯한 추가적인 분산 훈련 기법

[참조] https://homl.info/changes3

이 책의 학습 목표

1부 머신러닝

1장: 한눈에 보는 머신러닝	데이터 과학자가 꼭 알아야 할 기초 개념과 용어
2장: 머신러닝 프로젝트 처음부터 끝까지	주택 가격을 예측하는 회귀 작업을 살펴보면서 선형 회귀, 결정 트리, 랜덤 포레스트 등 여러 알고리즘 학습
3장: 분류	분류 시스템 학습
4장: 모델 훈련	신경망 구축에 필요한 모델 훈련 알고리즘 학습
5장: 서포트 벡터 머신	SVM의 핵심 개념, 사용 방법, 작동 원리 학습
6장: 결정 트리	결정 트리의 훈련, 시각화, 예측 방법과 사이킷런의 CART 훈련 알고리즘 등
7장: 앙상블 학습과 랜덤 포레스트	투표 기반 분류기, 배깅과 페이스팅 앙상블, 랜덤 포레스트, 부스팅, 스태킹 앙상블 등
8장: 차원 축소	고차원 공간과 차원 축소 기법
9장: 비지도 학습	비지도 학습과 알고리즘

이 책의 학습 목표

2부 신경망과 딥러닝

10장: 케라스를 사용한 인공 신경망 소개	인공 신경망과 케라스를 이용한 구현 방법
11장: 심층 신경망 훈련	심층 신경망의 문제와 해결 방법
12장: 텐서플로를 사용한 사용자 정의 모델과 훈련	텐서플로와 저수준 파이썬 API를 이용한 사용자 정의 모델과 훈련 알고리즘
13장: 텐서플로를 사용한 데이터 적재와 전처리	tf.data API와 TFRecord 포맷, 케라스 전처리 층 및 tf.data API 사용 방법
14장: 합성곱 신경망을 사용한 컴퓨터 비전	CNN 구성 요소, 케라스를 사용한 구현 방법
15장: RNN과 CNN을 사용한 시퀀스 처리	RNN 개념과 WaveNet 구현
16장: RNN과 어텐션을 사용한 자연어 처리	문장 수준의 RNN과 어텐션 메커니즘
17장: 오토인코더, GAN 그리고 확산 모델	오토인코더 차원 축소, 특성 추출, 비지도 사전 훈련 방법과 생성 모델
18장: 강화 학습	강화 학습 개념, 정책 그레이디언트, 심층 Q-네트워크
19장: 대규모 텐서플로 모델 훈련과 배포	TF 서빙과 구글 버텍스 AI 플랫폼에 모델을 배포하는 방법

이 책의 학습 목표

3부 부록

부록 A: 연습문제 정답

부록 B: 머신러닝 프로젝트 체크리스트

부록 C: 자동 미분

부록 D: 특수한 데이터 구조

부록 E: 텐서플로 그래프

Contents

- 9장: 비지도 학습

9.1 군집
9.2 가우스 혼합

9장 비지도 학습

비지도 학습과 알고리즘

이 장에서 다루는 비지도 학습 알고리즘

- 군집(clustering)
  - 비슷한 샘플을 클러스터(cluster)로 모음
  - 데이터 분석, 고객 분류, 추천 시스템, 검색 엔진, 이미지 분할, 준지도 학습, 차원 축소 등에 사용하는 훌륭한 도구
- 이상치 탐지(outlier detection)
  - '정상' 데이터가 어떻게 보이는지를 학습하고, 비정상 샘플을 감지하는 데 사용
  - 예를 들면 제조 라인에서 결함 제품을 감지하거나 시계열 데이터에서 새로운 트렌드를 찾음
- 밀도 추정(density estimation)
  - 데이터셋 생성 확률 과정(random process)의 확률 밀도 함수(probability density function, PDF)를 추정
  - 밀도 추정은 이상치 탐지에 널리 사용
  - 밀도가 매우 낮은 영역에 놓인 샘플이 이상치일 가능성이 높음
  - 데이터 분석과 시각화에도 유용

9.1 군집(1)

- 군집은 다양한 애플리케이션에서 사용
  - 고객 분류
  - 데이터 분석
  - 차원 축소 기법
  - 특성 공학
  - 이상치 탐지
  - 준지도 학습
  - 검색 엔진
  - 이미지 분할

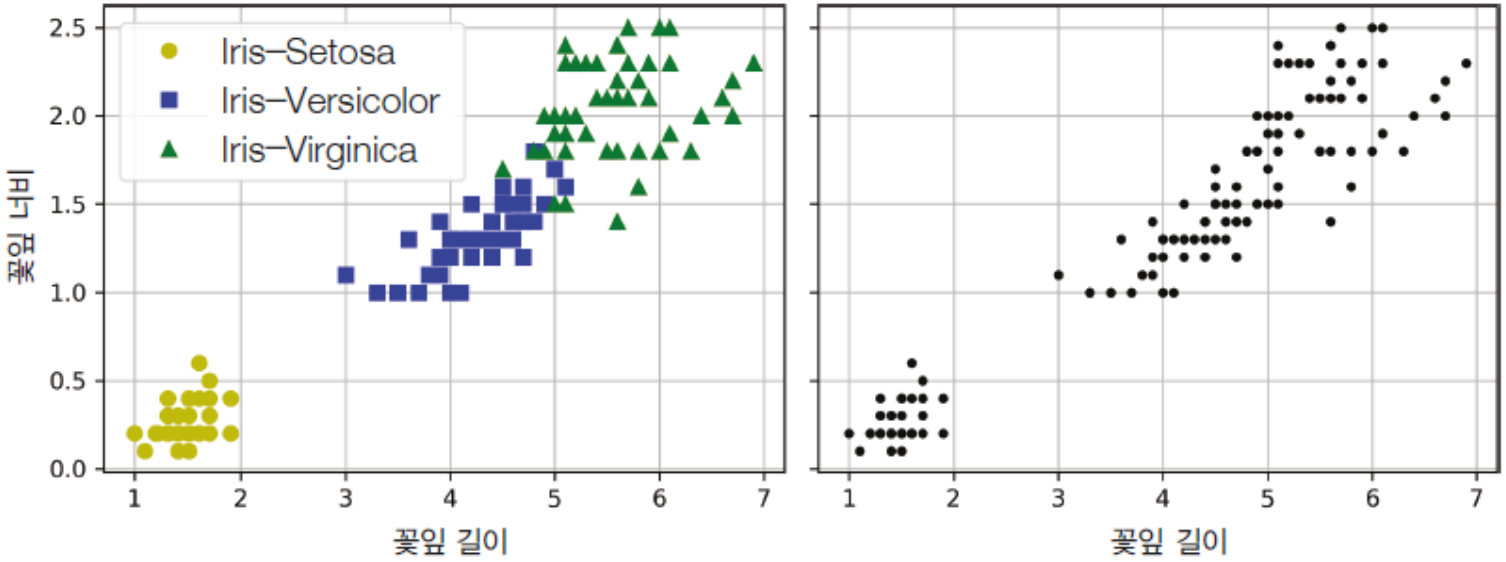

그림 9-1 분류(왼쪽) vs. 군집(오른쪽)

9.1 군집(2)

9.1.1 k -평균

- k-평균은 반복 몇 번으로 데이터셋을 빠르고 효율적으로 클러스터로 묶을 수 있는 간단한 알고리즘
  - 로이드-포지 알고리즘

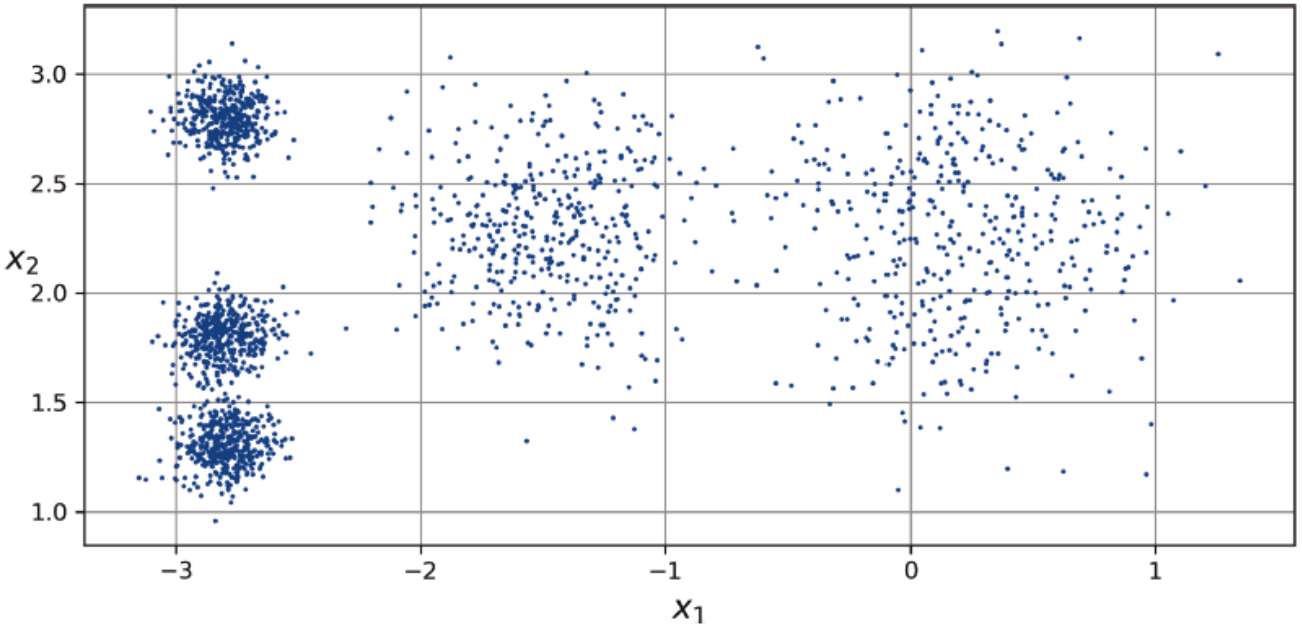

그림 9-2 샘플 덩어리 다섯 개로 이루어진 레이블 없는 데이터셋

9.1 군집(3)

- k-평균 알고리즘 훈련
  - 각 클러스터의 중심을 찾고 가장 가까운 클러스터에 샘플을 할당

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y = make_blobs([...]) # y는 클러스터 ID를 포함합니다.
                         # 하지만 이를 사용하지 않고 예측해보겠습니다.
k = 5
kmeans = KMeans(n_clusters=k, random_state=42)
y_pred = kmeans.fit_predict(X)
```

  - KMeans 클래스의 인스턴스는 labels_ 인스턴스 변수에 훈련된 샘플의 예측 레이블을 가지고 있음

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
>>> y_pred is kmeans.labels_
True
```

9.1 군집(4)

- 이 알고리즘이 찾은 다섯 개의 센트로이드

```
>>> kmeans.cluster_centers_
array([[-2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

- 새로운 샘플에 가장 가까운 센트로이드의 클러스터를 할당

```
>>> import numpy as np
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)
array([1, 1, 2, 2], dtype=int32)
```

9.1 군집(5)

- 클러스터의 결정 경계(보로노이 다이어그램Voronoi tessellation)
  - 센트로이드가 ×로 표시
  - k-평균 알고리즘은 클러스터의 크기가 많이 다르면 잘 작동하지 않음
    - 샘플을 클러스터에 할당할 때 센트로이드까지 거리를 고려하는 것이 전부이기 때문임

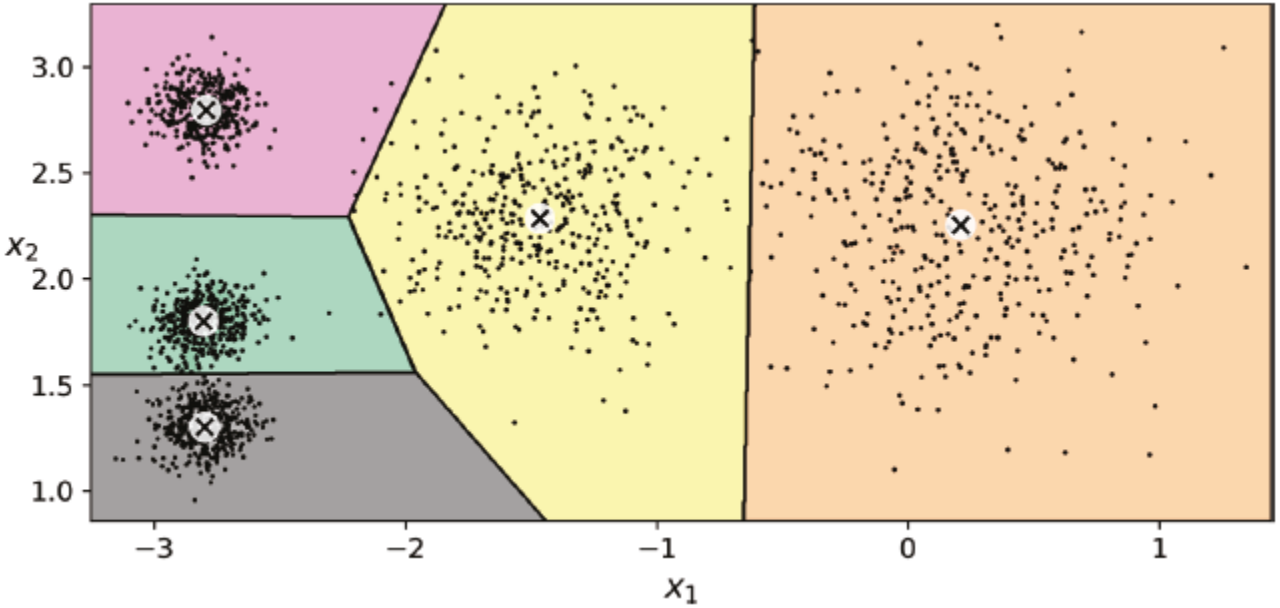

그림 9-3 k-평균의 결정 경계(보로노이 다이어그램)

9.1 군집(6)

- 하드 군집(hard clustering)
  - 샘플을 하나의 클러스터에 할당
- 소프트 군집(soft clustering)
  - 클러스터마다 샘플에 점수를 부여
- KMeans 클래스의 transform() 메서드는 샘플과 각 센트로이드 사이의 거리를 반환

```
>>> kmeans.transform(X_new).round(2)
array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```

9.1 군집(7)

k -평균 알고리즘

- 센트로이드를 랜덤하게 선정
  - (예를 들어 랜덤으로 데이터셋에서 k개의 샘플을 뽑아 그 위치를 센트로이드로 정함)
- 샘플에 레이블을 할당하고 센트로이드를 업데이트하고, 샘플에 레이블을 할당하고 센트로이드를 업데이트하는 식으로 센트로이드에 변화가 없을 때까지 계속
  - 이 알고리즘은 제한된 횟수 안에 수렴하는 것을 보장(일반적으로 이 횟수는 매우 적음)
  - 샘플과 가장 가까운 센트로이드 사이의 평균 제곱 거리는 각 단계마다 내려갈 수만 있고 음수가 될 수 없기 때문에 수렴이 보장됨

9.1 군집(8)

- [그림 9-4] 반복 세 번 만에 이 알고리즘은 최적으로 보이는 클러스터에 도달

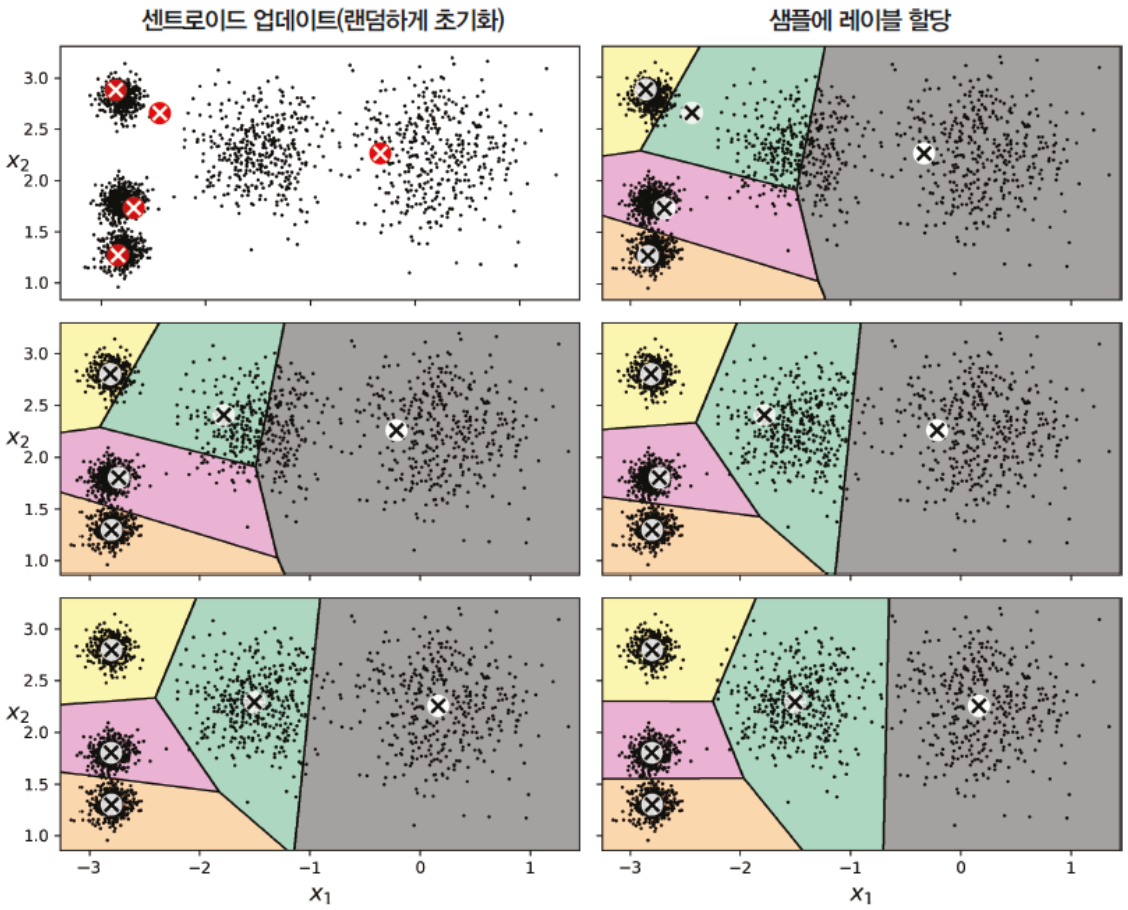

그림 9-4 k-평균 알고리즘

9.1 군집(9)

- 알고리즘의 수렴은 보장되지만 적절한 솔루션으로 수렴하지 못할 수 있음
  - 즉, 지역 최적점으로 수렴
  - 이 여부는 센트로이드 초기화에 달려있음

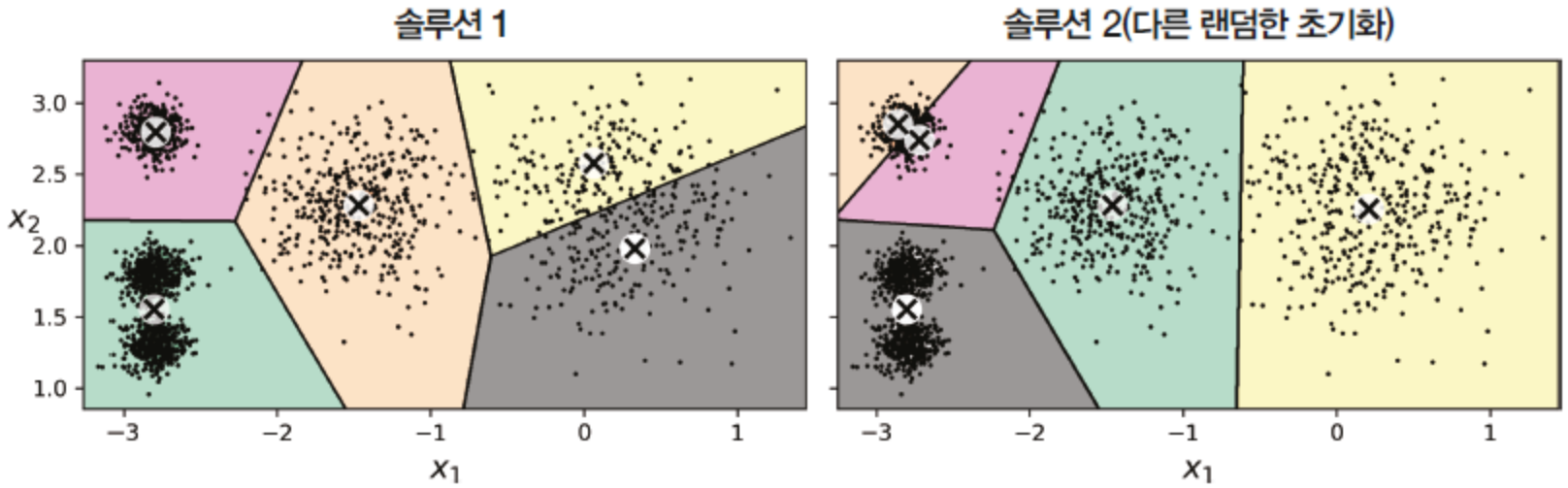

그림 9-5 운 나쁜 센트로이드 초기화 때문에 만들어진 최적이 아닌 솔루션

9.1 군집(10)

센트로이드 초기화 방법

- 센트로이드 위치를 근사하게 알 수 있다면(예를 들어 또 다른 군집 알고리즘을 먼저 실행) init 매개변수에 센트로이드 리스트를 담은 넘파이 배열을 지정하고 n_init을 1로 설정

```
good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
kmeans.fit(X)
```

- 랜덤 초기화를 다르게 하여 여러 번 알고리즘을 실행하고 가장 좋은 솔루션을 선택
  - 랜덤 초기화 횟수는 n_init 매개변수로 조절
  - 모델의 이너셔(inertia) – 성능 지표, 각 샘플과 가장 가까운 센트로이드 사이의 제곱 거리 합

```
>>> kmeans.inertia_
211.59853725816856
```

- score() 메서드는 이너셔의 음숫값을 반환

```
>>> kmeans.score(X)
-211.59853725816856
```

9.1 군집(11)

- k-평균++ 초기화 알고리즘
  1. 데이터셋에서 랜덤으로 균등하게 하나의 센트로이드 $c^{(1)}$을 선택
  2. $D(x^{(i)})^2 / \sum_{j=1}^{m} D\left(x^{(j)}\right)^2$ 의 확률로 샘플 $x^{(i)}$를 새로운 센트로이드 $c^{(i)}$로 선택
     $D(x^{(i)})$는 샘플 $x^{(i)}$와 이미 선택된 가장 가까운 센트로이드까지 거리
     이 확률 분포는 이미 선택한 센트로이드에서 멀리 떨어진 샘플을 다음 센트로이드로 선택할 가능성을 높임
  3. k개의 센트로이드가 선택될 때까지 이전 단계를 반복

9.1 군집(12)

k-평균 속도 개선과 미니배치 k-평균

- 전체 데이터셋을 사용해 반복하지 않고 각 반복마다 미니배치를 사용해 센트로이드를 조금씩 이동
- 사이킷런 MiniBatchKMeans 클래스

```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5, random_state=42)
minibatch_kmeans.fit(X)
```

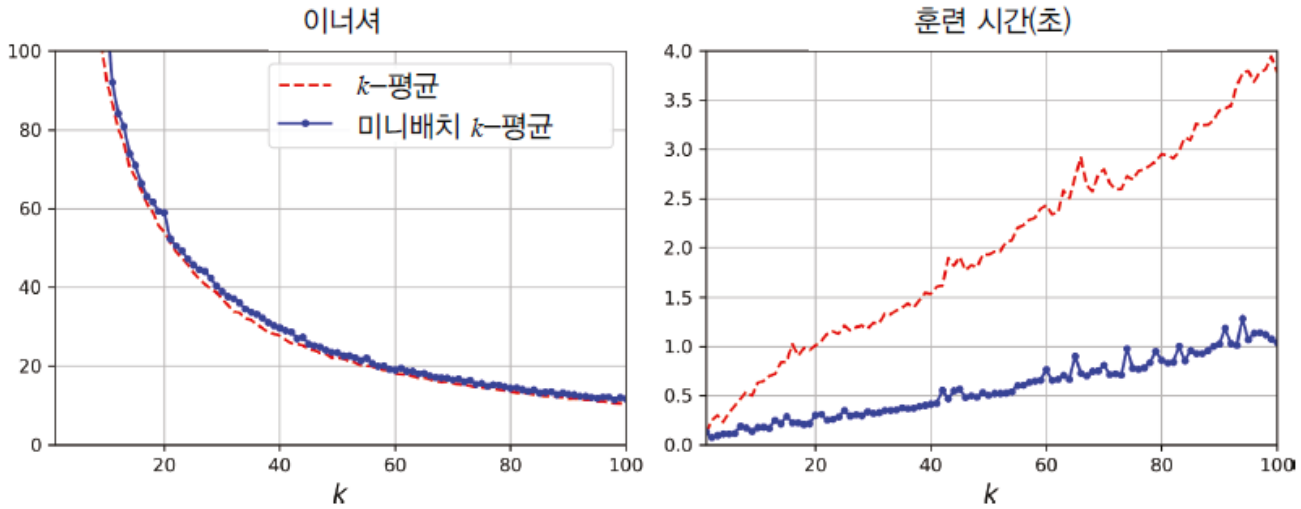

그림 9-6 미니배치 k-평균의 이너셔가 k-평균보다 높음(왼쪽)
하지만 훨씬 빠르고 k가 증가할수록 더 그러함(오른쪽)

9.1 군집(13)

최적의 클러스터 개수 찾기

- 올바르지 않은 클러스터 개수는 나쁜 모델을 만들게 됨

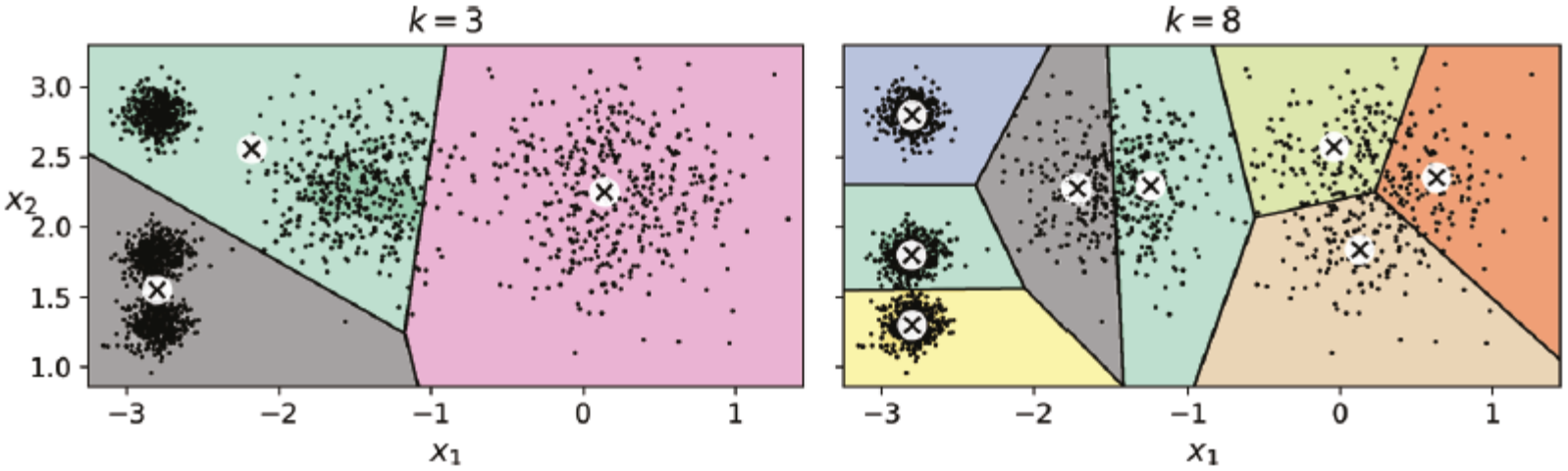

그림 9-7 잘못된 클러스터 개수 선택: k가 너무 작으면 별개의 클러스터가 합쳐지고(왼쪽), k가 너무 크면 하나의 클러스터가 여러 개로 나뉨(오른쪽)

9.1 군집(14)

최적의 클러스터 개수 찾기

- 가장 작은 이너셔를 가진 모델을 선택하면?
  - 이너셔는 k가 증가함에 따라 점점 작아지므로 k를 선택할 때 좋은 성능 지표가 아님

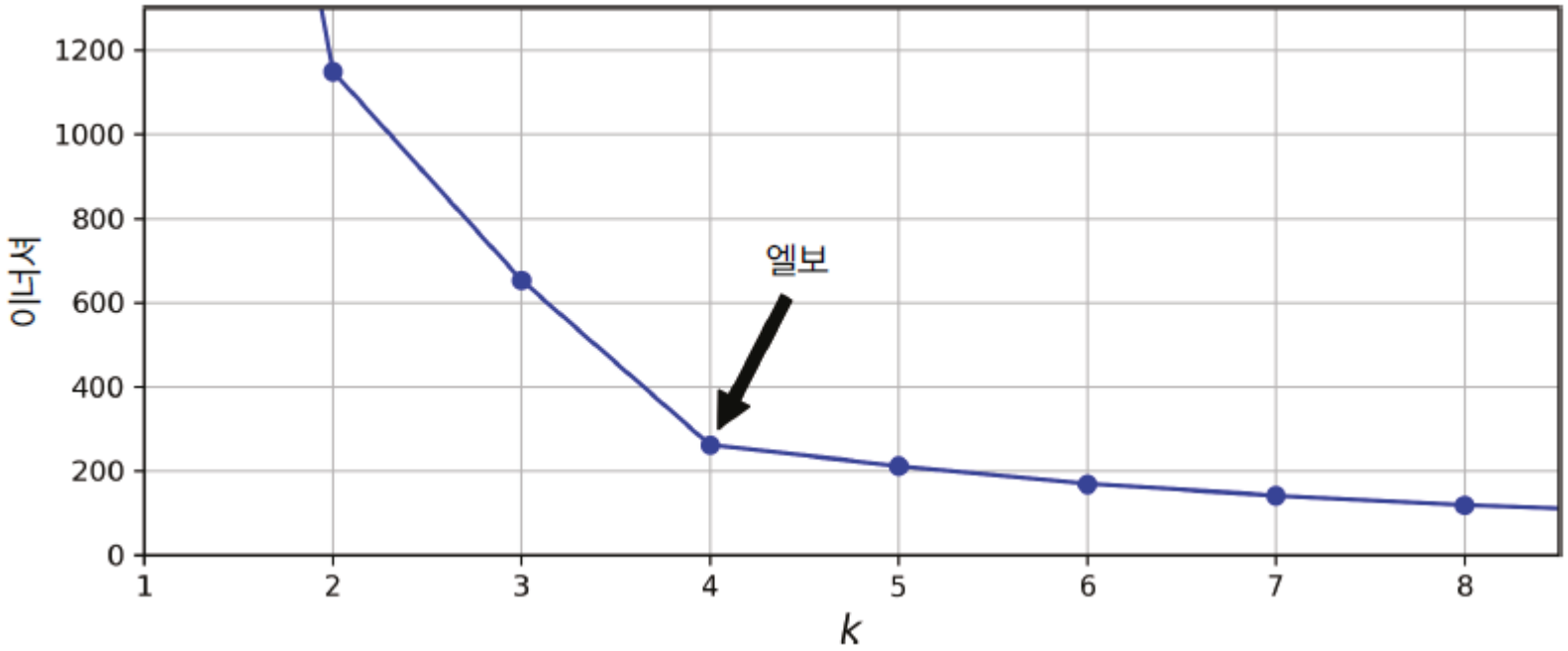

그림 9-8 클래스터 개수 k의 함수로 그린 이너셔 그래프

9.1 군집(15)

최적의 클러스터 개수 찾기

- 실루엣 점수(silhouette score) 사용
  - 모든 샘플에 대한 실루엣 계수(silhouette coefficient)의 평균

```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
0.655517642572828
```

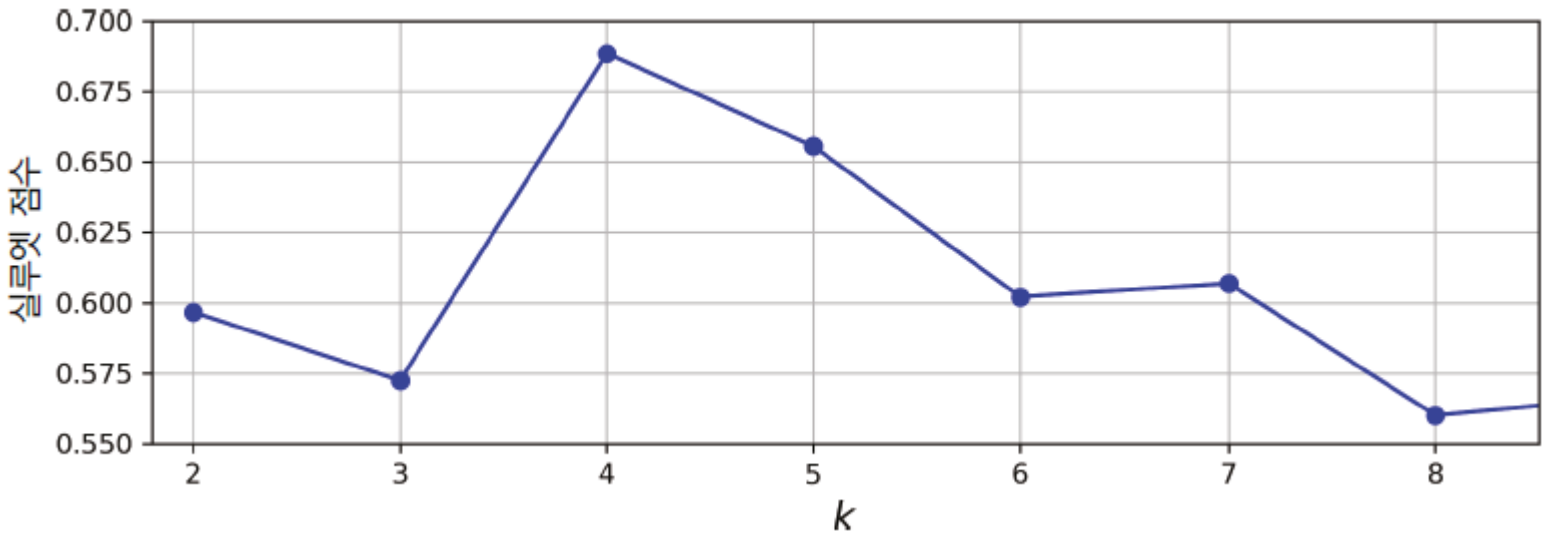

그림 9-9 실루엣 점수를 사용해 클러스터 개수 k를 선택하기

9.1 군집(16)

최적의 클러스터 개수 찾기

- 실루엣 다이어그램(silhouette diagram) 분석
  - 모든 샘플의 실루엣 계수를 할당된 클러스터와 계수 값으로 정렬하여 그리면 더 많은 정보가 있는 그래프

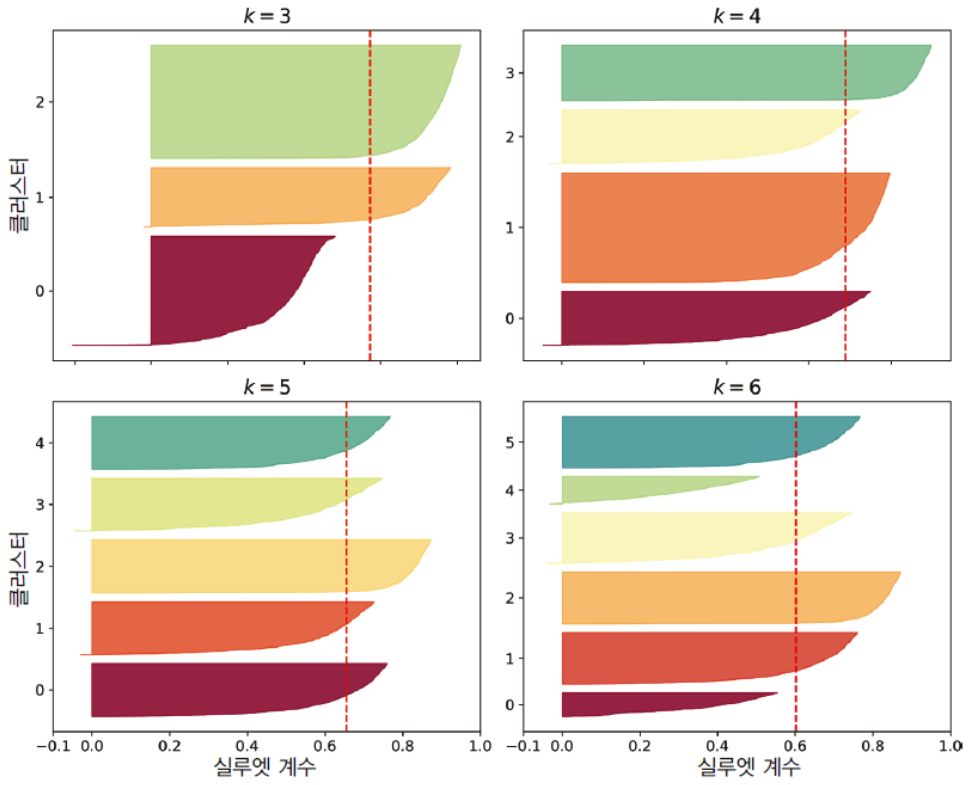

그림 9-10 여러 가지 k 값에 대한 실루엣 다이어그램 분석

9.1 군집(17)

9.1.2 k-평균의 한계

- k-평균은 장점
  - 속도가 빠르고 확장이 용이
- k-평균의 단점
  - 최적이 아닌 솔루션을 피하려면 알고리즘을 여러 번 실행해야 함
  - 클러스터 개수를 지정해야 함
  - k-평균은 클러스터의 크기 또는 밀집도가 서로 다르거나 원형8이 아닐 경우 잘 작동하지 않음

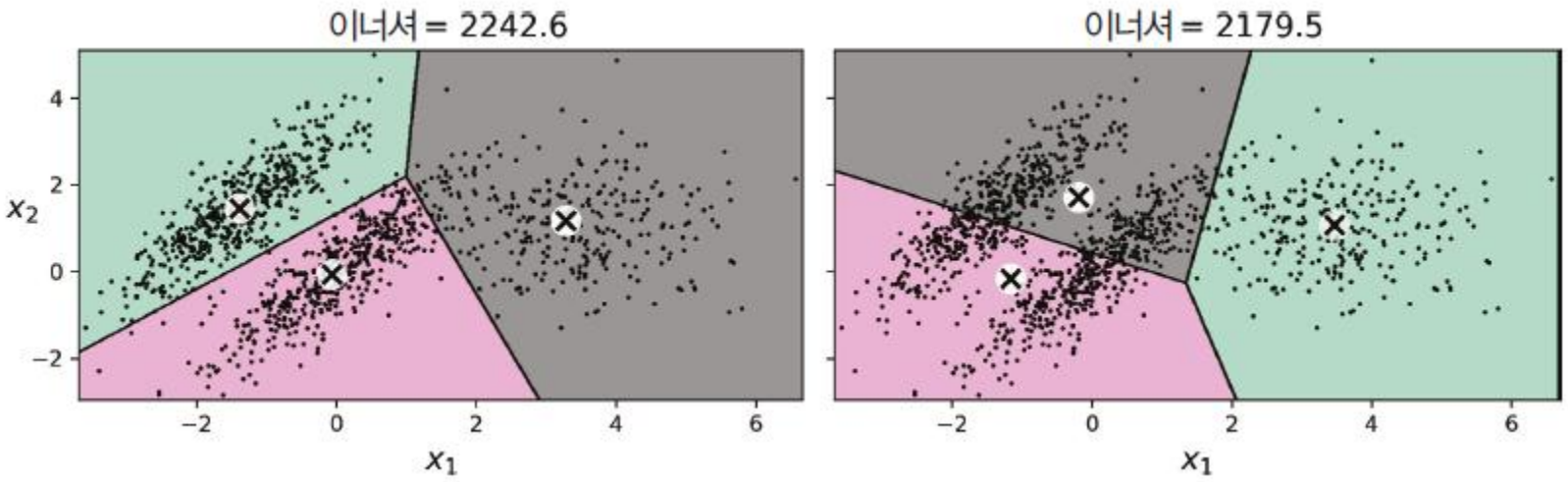

그림 9-11 k-평균이 세 개의 타원형 클러스터를 적절히 구분하지 못함

9.1 군집(18)

9.1.3 군집을 사용한 이미지 분할

- 이미지 분할(image segmentation)
  - 이미지를 여러 개의 세그먼트(segment)로 분할하는 작업

  - 색상 분할
    - 동일한 색상을 가진 픽셀을 같은 세그먼트에 할당
    - 많은 애플리케이션에서는 이 정도로 충분
    - 예를 들어 인공위성 사진을 분석하여 한 지역의 전체 산림 면적이 얼마나 되는지 측정하려면 색상 분할로 충분
  - 시맨틱 분할
    - 동일한 종류의 물체에 속한 모든 픽셀을 같은 세그먼트에 할당
    - 예를 들어 자율 주행 자동차의 비전 시스템에서 보행자 이미지를 구성하는 모든 픽셀은 '보행자' 세그먼트에 할당
  - 인스턴스 분할
    - 개별 객체에 속한 모든 픽셀을 같은 세그먼트에 할당
    - 이 경우 각 보행자는 다른 세그먼트가 됨

9.1 군집(19)

- k-평균을 사용하는 (훨씬 간단한) 색상 분할
  - Pillow 패키지(파이썬 이미지 처리 라이브러리인 PIL의 후속 버전)를 임포트한 다음, 이를 사용해 ladybug.png 이미지 ([그림 9-12]의 왼쪽 상단 이미지 참고)를 로드

```
>>> import PIL
>>> image = np.asarray(PIL.Image.open(filepath))
>>> image.shape
(533, 800, 3)
```

  - 배열의 크기를 바꾸어 긴 RGB 색상 리스트로 만든 다음 k-평균을 사용하여 8개의 클러스터로 모음
    - [그림 9-12]의 오른쪽 위에 보이는 이미지를 출력

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8, random_state=42).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

9.1 군집(20)

- [그림 9-12] 클러스터 개수를 여러 개로 바꿔 테스트

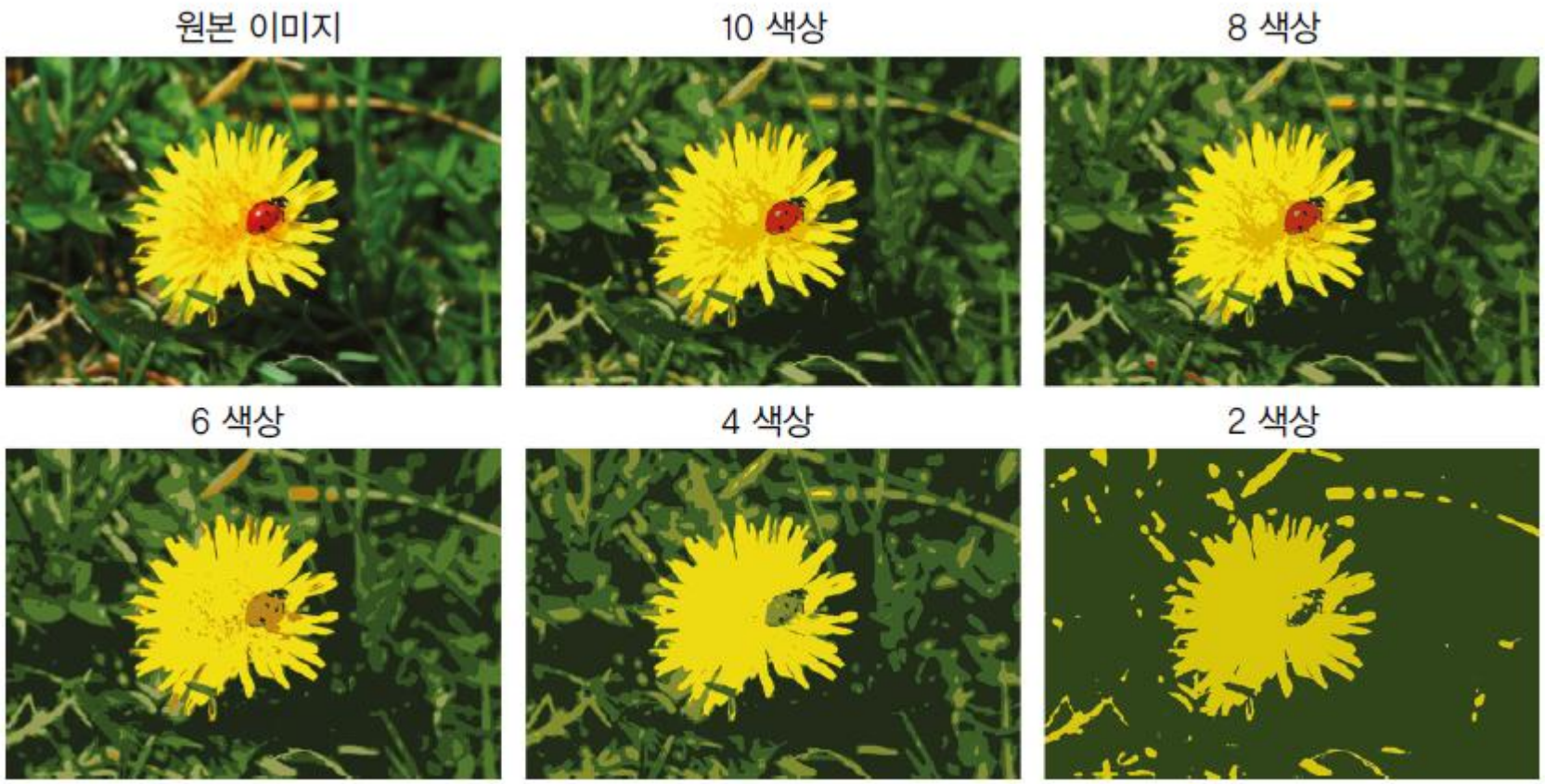

그림 9-12 다양한 클러스터 개수로 k-평균을 사용해 만든 이미지 분할

9.1 군집(21)

9.1.4 군집을 사용한 준지도 학습

- 준지도 학습은 레이블이 없는 데이터가 많고 레이블이 있는 데이터는 적을 때 사용
- 숫자 0에서 9까지를 나타내는 8×8 흑백 이미지 1,797개로 구성된 MNIST와 유사한 간단한 숫자 데이터셋을 사용
  - 데이터셋을 로드하고 분할

```
from sklearn.datasets import load_digits

X_digits, y_digits = load_digits(return_X_y=True)
X_train, y_train = X_digits[:1400], y_digits[:1400]
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

  - 50개 샘플에 대한 레이블만 있다고 가정하고 로지스틱 회귀 모델을 훈련

```
from sklearn.linear_model import LogisticRegression

n_labeled = 50
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

9.1 군집(22)

- 테스트 세트에서 이 모델의 정확도를 측정
  - 모델의 정확도는 74.8%

```
>>> log_reg.score(X_test, y_test)
0.7481108312342569
```

- 정확도 개선
  - 먼저 훈련 세트를 50개의 클러스터로 모으고 각 클러스터에서 센트로이드에 가장 가까운 이미지(대표 이미지)를 찾음

```
k = 50
kmeans = KMeans(n_clusters=k, random_state=42)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
X_representative_digits = X_train[representative_digit_idx]
```

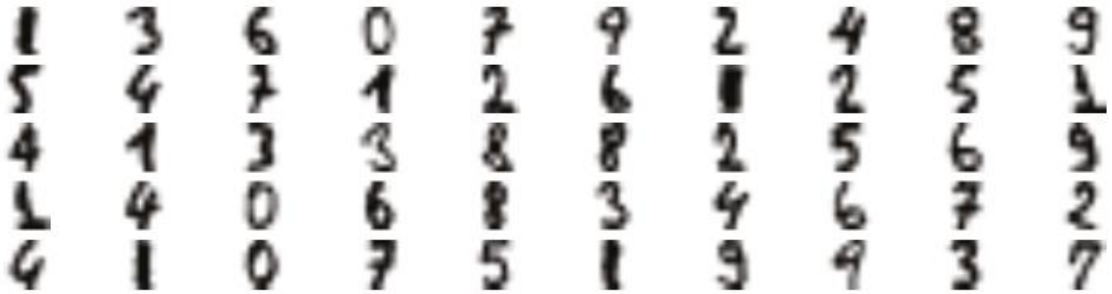

그림 9-13 50개의 대표 숫자 이미지(클러스터당 한 개)

9.1 군집(23)

- 이미지를 보고 수동으로 레이블을 할당

```
y_representative_digits = np.array([1, 3, 6, 0, 7, 9, 2, ..., 5, 1, 9, 9, 3, 7])
```

- 레이블된 50개의 샘플로 이루어진 데이터셋의 성능 확인

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.8488664987405542
```

- 레이블 전파(label propagation) - 레이블을 동일한 클러스터에 있는 모든 샘플로 전파

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
```

9.1 군집(24)

- 모델을 다시 훈련하고 성능을 확인

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.8942065491183879
```

- 클러스터 중심에서 가장 먼 1%의 샘플을 무시하여 일부 이상치 제거

```
percentile_closest = 99

X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1

partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train_propagated[partially_propagated]
```

9.1 군집(25)

- 모델을 다시 훈련하고 성능을 확인

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
>>> log_reg.score(X_test, y_test)
0.9093198992443325
```

- 레이블된 샘플 50개만으로(평균적으로 클래스당 5개 샘플) 90.9%의 정확도를 획득
- 이 레이블의 정확도는 다음 코드에서 볼 수 있듯이 약 97.5%에 달함

```
>>> (y_train_partially_propagated == y_train[partially_propagated]).mean()
0.9755555555555555
```

- 사이킷런은 레이블을 자동으로 전파할 수 있는 두 개의 클래스를 제공
- sklearn.semi_supervised패키지에 있는 LabelSpreading과 LabelPropagation

9.1 군집(26)

능동 학습

- 모델과 훈련 세트를 지속적으로 향상시키기 위해 다음 단계로 능동 학습(active learning)을 몇 번 반복
  - 이 방법은 전문가가 학습 알고리즘과 상호 작용하여 알고리즘이 요청할 때 특정 샘플의 레이블을 제공
- 능동 학습에서 가장 널리 사용되는 전략은 불확실성 샘플링(uncertainty sampling)
  1. 지금까지 수집한 레이블된 샘플에서 모델을 훈련
     이 모델을 사용해 레이블되지 않은 모든 샘플에 대한 예측을 만듦
  2. 모델이 가장 불확실하게 예측한 샘플(즉, 추정 확률이 낮은 샘플)을 전문가에게 보내 레이블을 붙임
  3. 레이블을 부여하는 노력만큼의 성능이 향상되지 않을 때까지 이를 반복

- 다른 능동 학습 전략은 모델을 가장 크게 바꾸는 샘플이나 모델의 검증 점수를 가장 크게 떨어뜨리는 샘플, 여러 개의 모델(예: SVM이나 랜덤 포레스트)이 동일한 예측을 내지 않는 샘플에 대해 레이블을 요청

9.1 군집(27)

9.1.5 DBSCAN

- DBSCAN(density-based spatial clustering of applications with noise)
  - 밀집된 연속적 지역을 클러스터로 정의
  - 작동 방식
    - 알고리즘이 각 샘플에서 작은 거리인 ε 내에 샘플이 몇 개 놓여 있는지 계산
      이 지역을 샘플의 ε–이웃(ε-neighborhood)이라고 함
    - (자기 자신을 포함해) ε-이웃 내에 적어도 min_samples개 샘플이 있다면 이를 핵심 샘플core instance로 간주
      즉, 핵심 샘플은 밀집된 지역에 있는 샘플
    - 핵심 샘플의 이웃에 있는 모든 샘플은 동일한 클러스터에 속함
      이웃에는 다른 핵심 샘플이 포함될 수 있음
      따라서 핵심 샘플의 이웃의 이웃은 계속해서 하나의 클러스터를 형성
    - 핵심 샘플이 아니고 이웃도 아닌 샘플은 이상치로 판단

9.1 군집(28)

- DBSCAN은 모든 클러스터가 밀집되지 않은 지역과 잘 구분될 때 좋은 성능을 발휘

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

- 모든 샘플의 레이블은 인스턴스 변수 labels_에 저장
  - 일부 샘플의 클러스터 인덱스는 -1. 이는 알고리즘이 이 샘플을 이상치로 판단했다는 의미

```
>>> dbscan.labels_
array([ 0,  2, -1, -1,  1,  0,  0,  0, ...,  3,  2,  3,  3,  4,  2,  6,  3])
```

9.1 군집(29)

- 핵심 샘플의 인덱스는 인스턴스 변수 core_sample_indices_에서 확인

```
>>> len(dbscan.core_sample_indices_)
808
>>> dbscan.core_sample_indices_
array([ 0,  4,  5,  6,  7,  8, 10, 11, ..., 992, 993, 995, 997, 998, 999])
>>> dbscan.components_
array([[-0.02137124,  0.40618608],
       [-0.84192557,  0.53058695],
                  ...
       [ 0.79419406,  0.60777171]])
```

9.1 군집(30)

- 군집 결과는 [그림 9-14]의 왼쪽 그래프에서 확인
  - 클러스터를 7개 만들었고 많은 샘플을 이상치로 판단
  - eps를 0.2로 증가시켜 샘플의 이웃 범위를 넓히면 오른쪽 그래프처럼 완벽한 군집을 형성

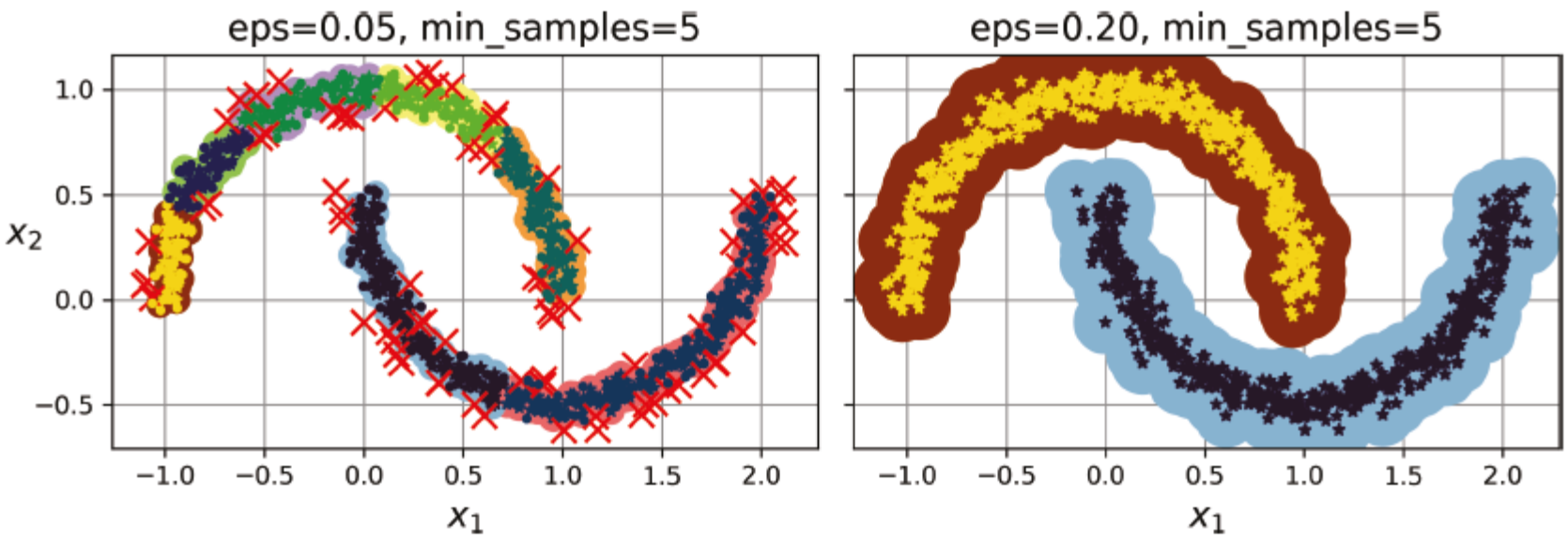

그림 9-14 두 가지 이웃 반경을 사용한 DBSCAN 군집

9.1 군집(31)

- DBSCAN 클래스는 predict() 메서드를 제공하지 않고 fit_predict() 메서드를 제공
  - 이 알고리즘은 새로운 샘플에 대해 클러스터를 예측할 수 없음
  - 따라서 사용자가 필요한 예측기를 선택
  - KNeighborsClassifier를 훈련

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

  - 샘플 몇 개를 전달하여 어떤 클러스터에 속할 가능성이 높은지 예측하고 각 클러스터에 대한 확률을 추정

```
>>> X_new = np.array([[-0.5, 0], [0, 0.5], [1, -0.1], [2, 1]])
>>> knn.predict(X_new)
array([1, 0, 1, 0])
>>> knn.predict_proba(X_new)
array([[0.18, 0.82],
       [1.  , 0.  ],
       [0.12, 0.88],
       [1.  , 0.  ]])
```

9.1 군집(32)

- 이 분류기를 핵심 샘플에서만 훈련했지만 모든 샘플에서 훈련할 수도 있음
  - 또는 이상치를 제외
  - 선택은 최종 작업의 성능에 따라 결정
  - KNeighborsClassifier의 kneighbors() 메서드를 사용

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()
array([-1,  0,  1, -1])
```

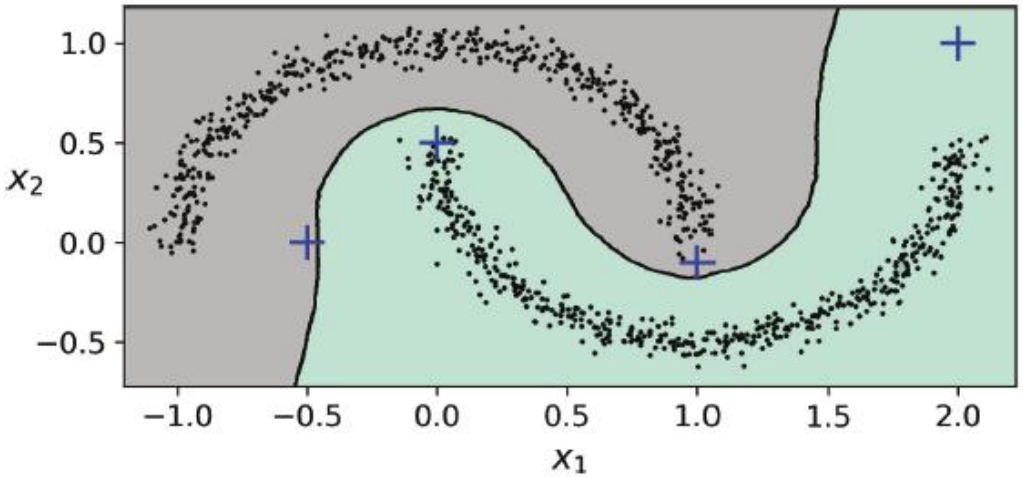

그림 9-15 두 클러스터 사이의 결정 경계

9.1 군집(33)

9.1.6 다른 군집 알고리즘

- 병합 군집
  - 클러스터 계층을 밑바닥부터 위로 쌓아 구성
- BIRCH(balanced iterative reducing and clustering using hierarchies)
  - 특별히 대규모 데이터셋을 위해 고안
- 평균-이동
  - 각 샘플을 중심으로 하는 원을 그리고 원마다 안에 포함된 모든 샘플의 평균을 구함
  - 원의 중심을 평균점으로 이동
  - 모든 원이 움직이지 않을 때까지 이러한 평균-이동(mean-shift)을 계속
- 유사도 전파
  - 모든 샘플은 자신을 대표할 다른 샘플(또는 자신)을 선택할 때까지 샘플 간에 메시지를 반복적으로 교환
  - 예시(exemplar) - 선출된 샘플
- 스펙트럼 군집
  - 샘플 사이의 유사도 행렬을 받아 저차원 임베딩을 만듦(즉, 행렬의 차원을 축소)
  - 이 저차원 공간에서 또 다른 군집 알고리즘을 사용

9.2 가우스 혼합(1)

- 가우스 혼합 모델(Gaussian mixture model, GMM)
  - 샘플이 파라미터가 알려지지 않은 여러 개의 혼합된 가우스 분포에서 생성되었다고 가정하는 확률 모델
  - GMM은 여러 변형이 있으며 가장 간단한 버전이 GaussianMixture 클래스에 구현
    - 사전에 가우스 분포의 개수 k를 알아야 함
    - 데이터셋 X가 다음 확률 과정을 통해 생성되었다고 가정
      - 샘플마다 k개의 클러스터에서 랜덤하게 한 클러스터가 선택
        j번째 클러스터를 선택할 확률은 클러스터의 가중치 $\phi^{(j)}$
        i번째 샘플을 위해 선택한 클러스터 인덱스는 $z^{(i)}$로 표시
      - i번째 샘플이 j번째 클러스터에 할당되었다면($z^{(i)}=j$) 이 샘플의 위치 $x^{(i)}$는 평균이 $\mu^{(j)}$이고 공분산 행렬이 $\Sigma^{(j)}$인 가우스 분포에서 랜덤하게 샘플링됨
        이를 $x^{i} \sim N(\mu^{(j)}, \Sigma^{(j)})$ 와 같이 씀

9.2 가우스 혼합(2)

- 데이터셋 X가 주어지면 가중치 ϕ, 전체 분포의 파라미터 $\mu^{(1)}$에서 $\mu^{(k)}$까지, $\Sigma^{(1)}$에서 $\Sigma^{(k)}$까지를 추정
  - 사이킷런의 GaussianMixture 클래스

```python
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10, random_state=42)
gm.fit(X)
```

  - 추정한 파라미터 확인

```
>>> gm.weights_
array([0.39025715, 0.40007391, 0.20966893])
>>> gm.means_
array([[ 0.05131611, 0.07521837],
       [-1.40763156, 1.42708225],
       [ 3.39893794, 1.05928897]])
>>> gm.covariances_
array([[[ 0.68799922, 0.79606357],
        [ 0.79606357, 1.21236106]],

       [[ 0.63479409, 0.72970799],
        [ 0.72970799, 1.1610351 ]],

       [[ 1.14833585, -0.03256179],
        [-0.03256179,  0.95490931]]])
```

기댓값-최대화(expectation-maximization, EM) 알고리즘

- 기댓값 단계(expectation step) - 샘플을 클러스터에 할당
- 최대화 단계(maximization step) - 클러스터를 업데이트

- 클러스터에 속할 추정 확률로 샘플에 가중치가 적용
  - 이 확률을 샘플에 대한 클러스터의 책임(responsibility)이라 함

9.2 가우스 혼합(3)

- 알고리즘이 수렴했는지 여부와 반복 횟수를 확인

```
>>> gm.converged_
True
>>> gm.n_iter_
4
```

- 새로운 샘플을 가장 비슷한 클러스터에 손쉽게 할당(하드 군집)
  또는 특정 클러스터에 속할 확률을 예측(소프트 군집)
  - 하드 군집을 위해서는 predict() 메서드를 사용하고 소프트 군집을 위해서는 predict_proba() 메서드를 사용

```
>>> gm.predict(X)
array([0, 0, 1, ..., 2, 2, 2])
>>> gm.predict_proba(X).round(3)
array([[0.977, 0. , 0.023],
       [0.983, 0.001, 0.016],
       [0. , 1. , 0. ],
       ...,
       [0. , 0. , 1. ],
       [0. , 0. , 1. ],
       [0. , 0. , 1. ]])
```

9.2 가우스 혼합(4)

- 가우스 혼합 모델은 생성 모델(generative model)
  - 즉, 이 모델에서 새로운 샘플을 만들 수 있음

```
>>> X_new, y_new = gm.sample(6)
>>> X_new
array([[-0.86944074, -0.32767626],
       [ 0.29836051, 0.28297011],
       [-2.8014927 , -0.09047309],
       [ 3.98203732, 1.49951491],
       [ 3.81677148, 0.53095244],
       [ 2.84104923, -0.73858639]])
>>> y_new
array([0, 0, 1, 2, 2, 2])
```

  - score_samples() 메서드를 사용하여 주어진 위치에서 모델의 밀도를 추정
    - 샘플이 주어지면 이 메서드는 그 위치의 확률 밀도 함수(PDF)의 로그를 예측

```
>>> gm.score_samples(X).round(2)
array([-2.61, -3.57, -3.33, ..., -3.51, -4.4 , -3.81])
```

9.2 가우스 혼합(5)

- [그림 9-16] 이 모델의 클러스터 평균, 결정 경계(파선), 밀도 등고선

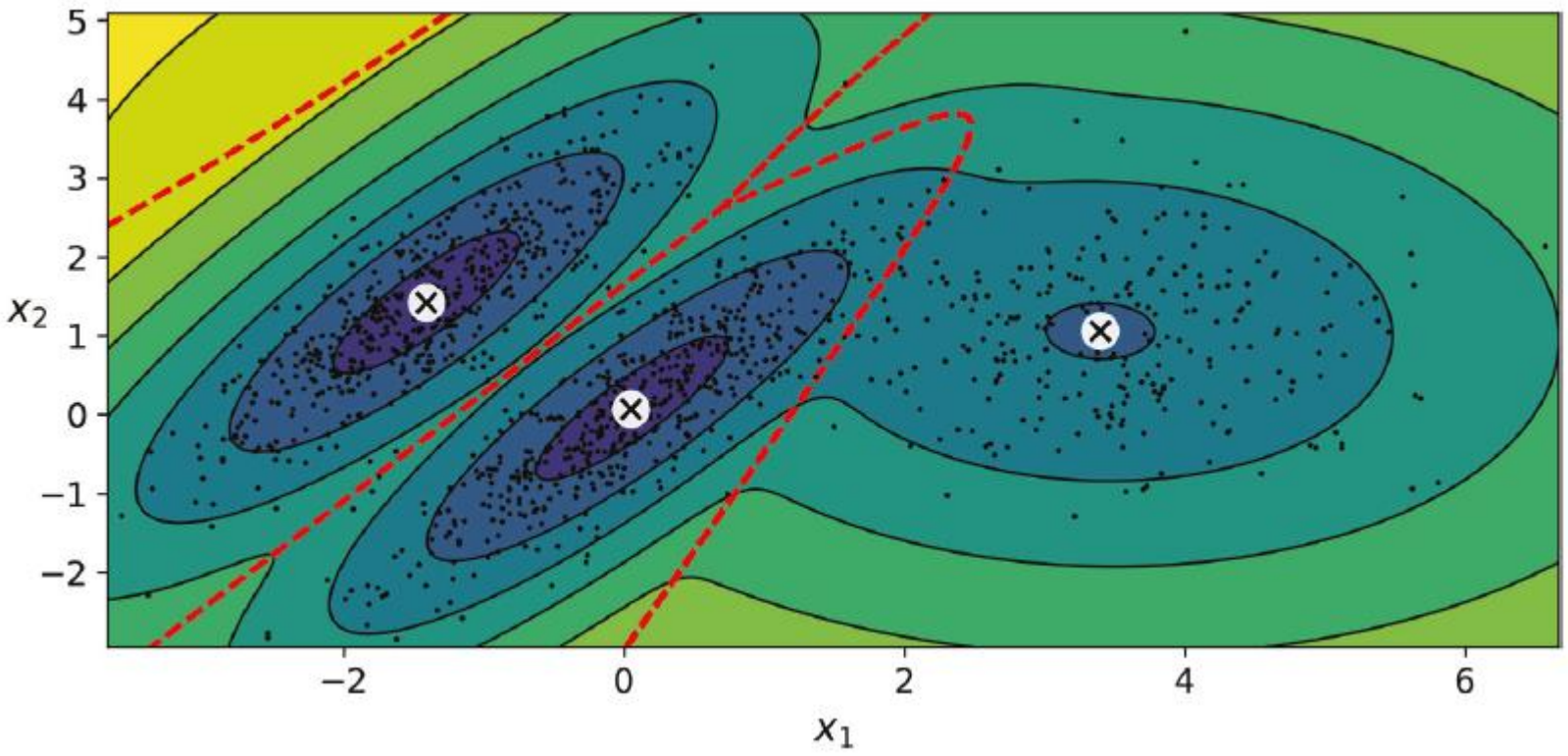

그림 9-16 훈련된 가우스 혼합 모델의 클러스터 평균, 결정 경계, 밀도 등고선

9.2 가우스 혼합(6)

- 특성이나 클러스터가 많거나 샘플이 적을 때는 EM이 최적의 솔루션으로 수렴하기 어려움
  - 알고리즘이 학습할 파라미터 개수를 제한
    - 클러스터의 모양과 방향의 범위를 제한
    - 공분산 행렬에 제약을 추가
  - 사이킷런에서는 covariance_type 매개변수에 다음 값 중 하나를 설정
    - "spherical"
      모든 클러스터가 원형
      하지만 지름은 다를 수 있음(즉, 분산이 다름)
    - "diag"
      클러스터는 크기에 상관없이 어떤 타원형도 가능
      하지만 타원의 축은 좌표축과 나란해야 함(즉, 공분산 행렬이 대각 행렬)
    - "tied"
      모든 클러스터가 동일한 타원 모양, 크기, 방향을 가짐(즉, 모든 클러스터는 동일한 공분산 행렬을 공유)

9.2 가우스 혼합(7)

- [그림 9-17] covariance_type을 "tied" 또는 "spherical"로 지정했을 때 EM 알고리즘으로 찾은 솔루션

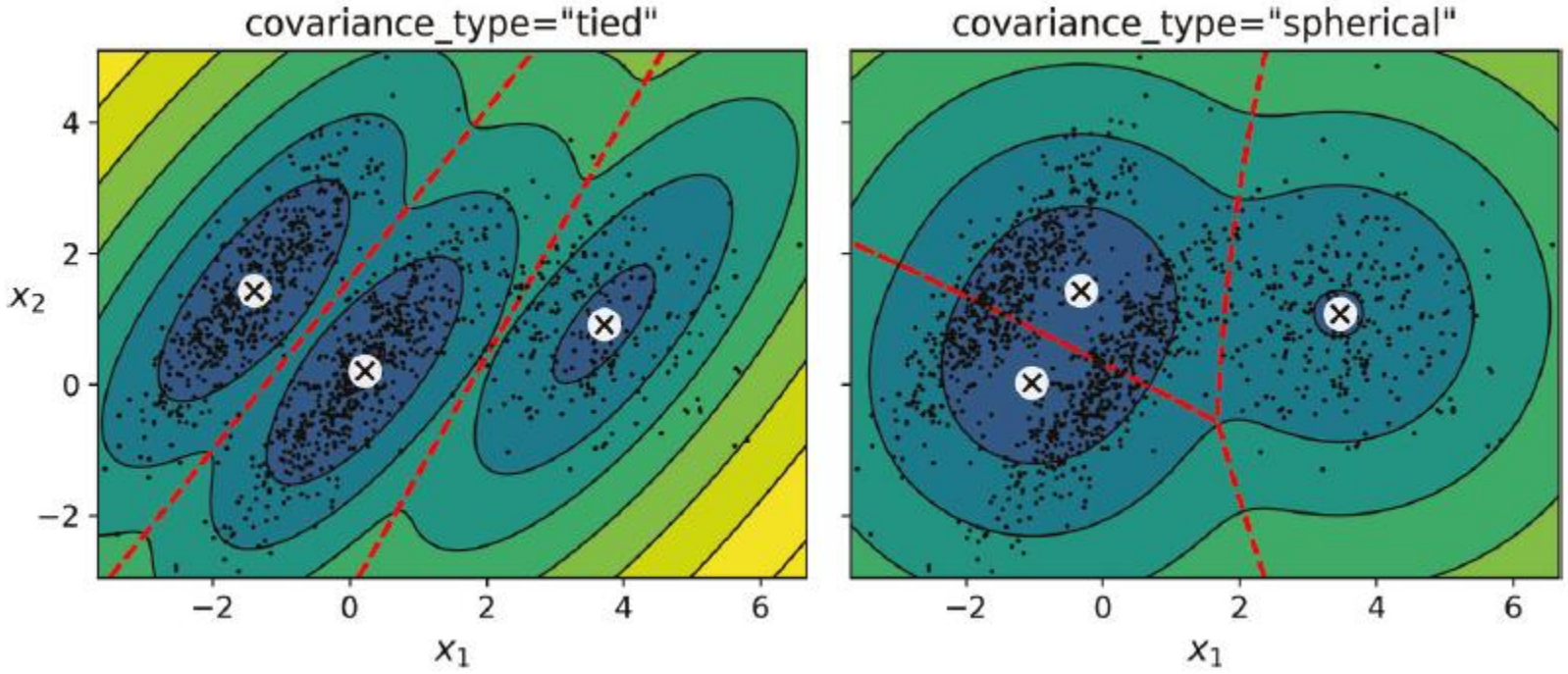

그림 9-17 타이드(tied) 클러스터(왼쪽)와 원형 클러스터(오른쪽)를 사용한 가우스 혼합

9.2 가우스 혼합(8)

9.2.1 가우스 혼합을 사용한 이상치 탐지

- 밀도가 낮은 지역에 있는 모든 샘플을 이상치로 간주
  - 네 번째 백분위수(4%)를 밀도 임곗값으로 사용하여 이상치를 구분하는 코드

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 2)
anomalies = X[densities < density_threshold]
```

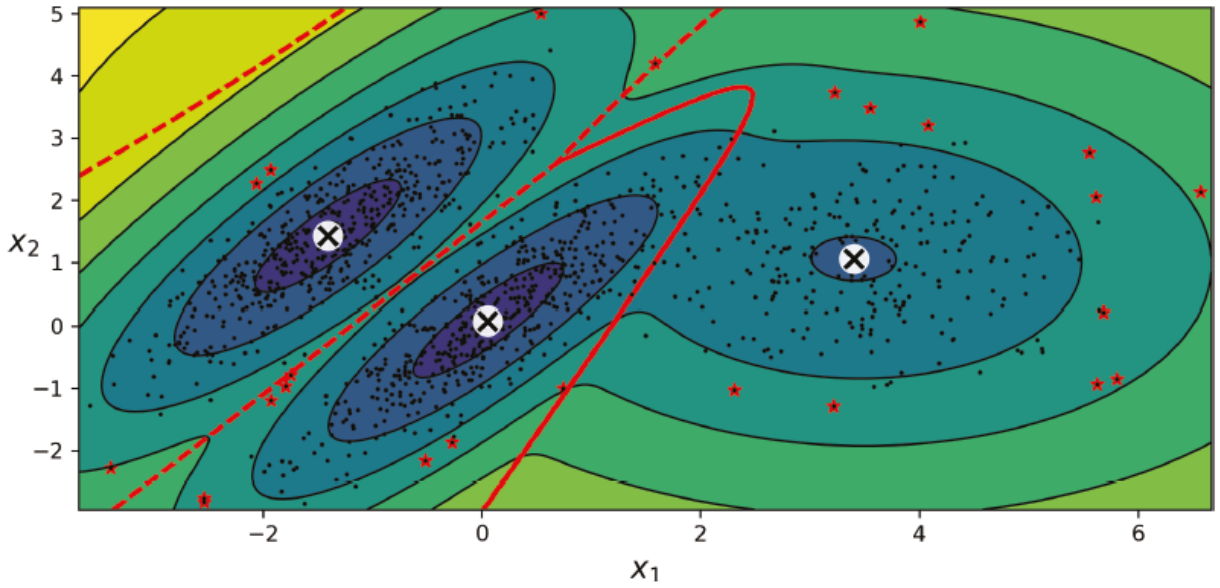

그림 9-18 가우스 혼합 모델을 사용한 이상치 탐지

특이치 탐지는 이상치로 오염되지 않은 '깨끗한' 데이터셋에서 훈련한다는 점이 이상치 탐지와 다름

9.2 가우스 혼합(9)

9.2.2 클러스터 개수 선택

- BIC나 AIC와 같은 이론적 정보 기준을 최소화하는 모델을 선택

식 9-1 BIC와 AIC

$$BIC = \log(m)\,p - 2\log(\hat{\mathscr{L}})$$

$$AIC = 2p - 2\log(\hat{\mathscr{L}})$$

- m은 샘플의 개수
- p는 모델이 학습할 파라미터 개수
- $\hat{\mathcal{L}}$은 모델의 가능도 함수(likelihood function)의 최댓값

9.2 가우스 혼합(10)

- bic()와 aic() 메서드를 사용해 BIC와 AIC를 계산

```
>>> gm.bic(X)
8189.747000497186
>>> gm.aic(X)
8102.521720382148
```

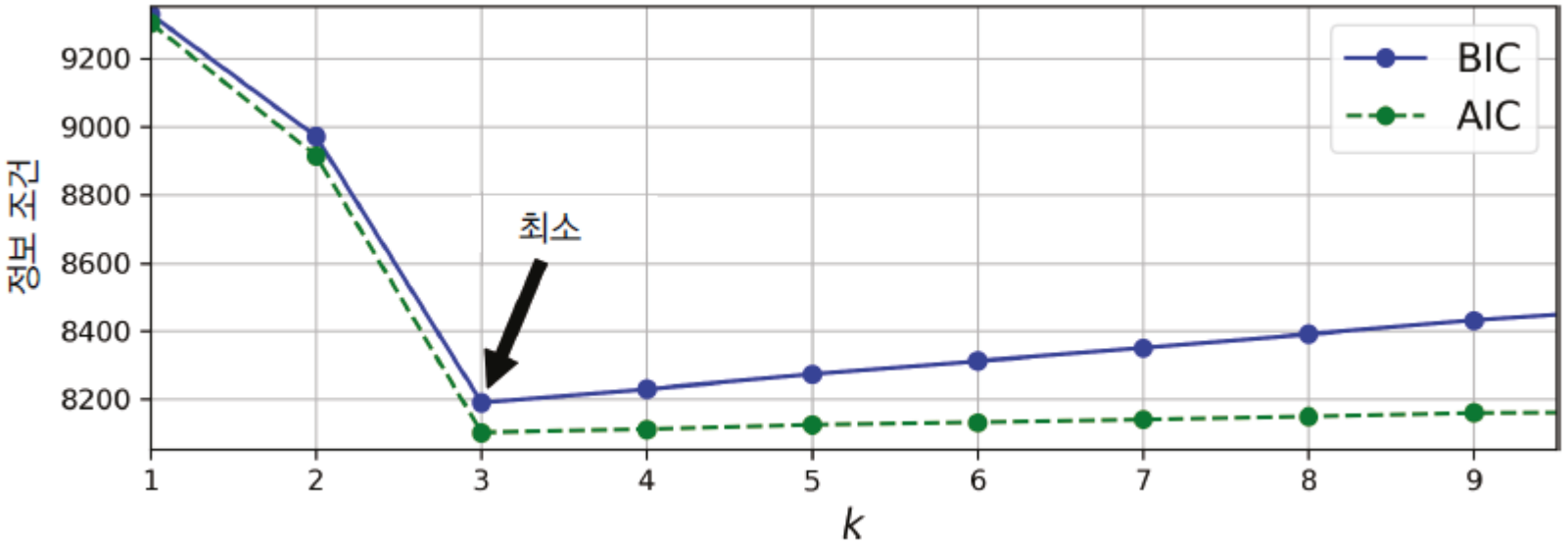

그림 9-20 여러 가지 클러스터 개수 k에 대한 AIC와 BIC

9.2 가우스 혼합(11)

9.2.3 베이즈 가우스 혼합 모델

- 최적의 클러스터 개수를 수동으로 찾지 않고 불필요한 클러스터의 가중치를 0으로(또는 0에 가깝게) 만드는 BayesianGaussianMixture 클래스
  - 자동으로 불필요한 클러스터를 제거
  - 클러스터 개수를 10으로 설정하고 결과를 확인

```
>>> from sklearn.mixture import BayesianGaussianMixture
>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10, random_state=42)
>>> bgm.fit(X)
>>> bgm.weights_.round(2)
array([0.4 , 0.21, 0.4 , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0. ])
```

그림 9-20 여러 가지 클러스터 개수 k에 대한 AIC와 BIC

9.2 가우스 혼합(12)

- 베이즈 가우스 혼합 모델은 타원형 클러스터에 잘 작동
  - 다른 모양을 가진 클러스터에서는 잘 작동하지 않음

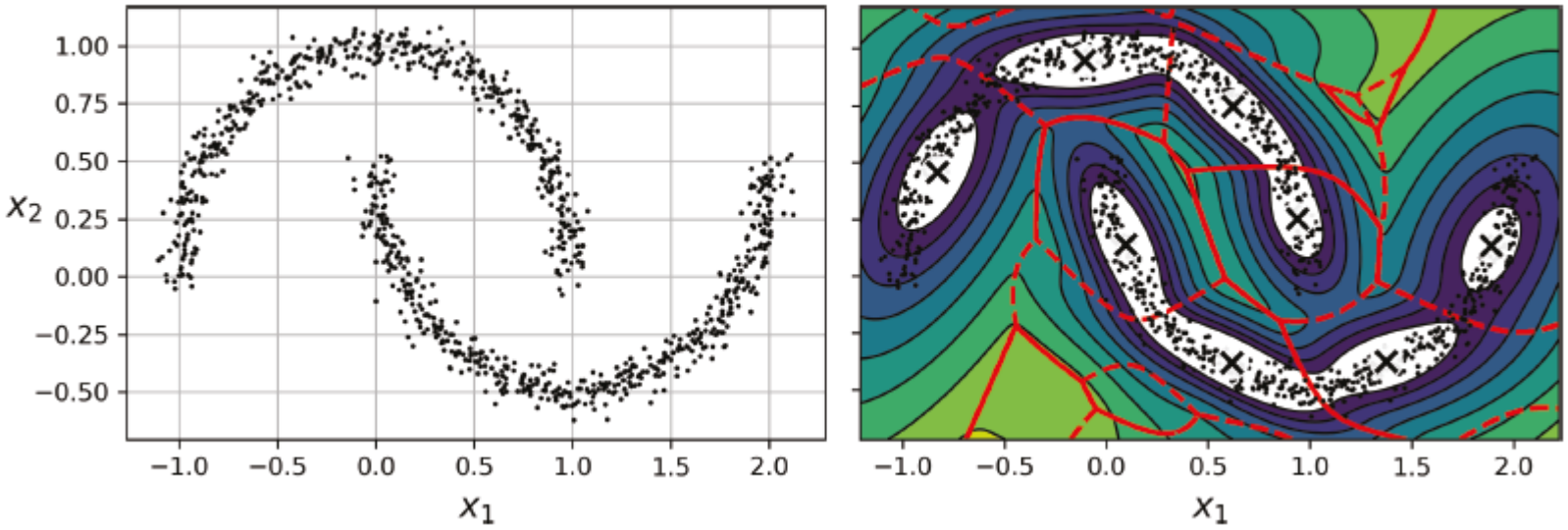

그림 9-21 타원형이 아닌 클러스터에 가우스 혼합 모델 훈련하기

9.2 가우스 혼합(13)

9.2.4 이상치 탐지와 특이치 탐지를 위한 알고리즘

- Fast-MCD
  - EllipticEnvelope 클래스에서 구현된 Fast-MCD(minimum covariance determinant) 알고리즘은 이상치 탐지에 유용
  - 특히 데이터셋을 정제할 때 사용
- 아이솔레이션 포레스트
  - 고차원 데이터셋에서 이상치 탐지에 효율적인 알고리즘
- LOF(local outlier factor)
  - 이상치 탐지에 좋음
  - 주어진 샘플 주위의 밀도와 이웃 주위의 밀도를 비교
- one-class SVM
  - 이 알고리즘은 특이치 탐지에 효과적
- PCA(그리고 inverse_transform() 메서드를 가진 다른 차원 축소 기법)
  - 이상치 탐지에 매우 효과적이며 간단함

연습문제(1)

1. 군집을 어떻게 정의할 수 있는가? 몇 개의 군집 알고리즘을 말해보기
2. 군집 알고리즘의 주요 애플리케이션은 무엇인가?
3. k-평균을 사용할 때 적절한 클러스터 개수를 선택할 수 있는 두 가지 기법은?
4. 레이블 전파는 무엇인가? 왜 이를 구현해야 하고 어떻게 구현할 수 있나?
5. 대규모 데이터셋으로 확장할 수 있는 군집 알고리즘 두 개를 말해보기. 밀도가 높은 지역을 찾는 군집 알고리즘 두 개는 무엇인가?
6. 능동 학습이 유용한 경우는 언제인가? 어떻게 구현할 수 있나?
7. 이상치 탐지와 정상치 탐지의 차이는 무엇인가?
8. 가우시안 혼합이 무엇인가요? 어떤 작업에 사용할 수 있나?
9. 가우시안 혼합 모델을 사용할 때 적절한 클러스터 개수를 찾는 두 가지 기법은?

연습문제(2)

10. 전통적인 올리베티(Olivetti) 얼굴 데이터셋은 64×64 픽셀 크기의 흑백 얼굴 이미지 400개를 담고 있음. 각 이미지는 4,096 크기의 1D 벡터로 펼쳐져 있음. 사람 40명의 사진을 10장씩 찍은 것임. 어떤 사람의 사진인지 예측하는 모델을 훈련하는 것이 일반적.
    sklearn.datasets.fetch_olivetti_faces() 함수를 사용해 데이터셋을 불러오고 훈련 세트, 검증 세트, 테스트 세트로 나누기(이 데이터셋은 이미 0에서 1 사이로 스케일이 조정되어 있음). 이 데이터셋은 매우 작으니 계층적 샘플링을 사용해 각 세트에 동일한 사람의 얼굴이 고루 섞이도록 하는 것이 좋음.
    그다음 k-평균을 사용해 이미지를 군집해보기. (이 장에서 소개한 기법 중 하나를 사용해) 적절한 클러스터 개수를 찾고, 클러스터를 시각화해보기.
    각 클러스터에 비슷한 얼굴이 들어 있나?

11. 올리베티 얼굴 데이터셋으로 계속.
    사진에 나타난 사람을 예측하는 분류기를 훈련하고 검증 세트에서 평가해보고,. 그다음 k-평균을 차원 축소 도구로 사용하여 축소된 세트에서 분류기를 훈련해보기.
    분류기 성능을 최대로 만드는 클러스터 개수를 찾아보면, 얼마나 성능이 나오나?
    축소된 세트에서 추출한 특성을 원본 특성에 추가하면 어떤가? (여기에서도 최선의 클러스터 개수를 찾아보기)

연습문제(3)

12. 올리베티 얼굴 데이터셋에서 가우시안 혼합 모델을 훈련해보기
    알고리즘의 속도를 높이기 위해 데이터셋의 차원을 감소시켜야 할 것임(예를 들면 분산의 99%를 유지하면서 PCA를 사용).
    이 모델을 사용해 (sample() 메서드로) 새로운 얼굴을 생성하고 시각화해보기(PCA를 사용했다면 inverse_transform() 메서드를 사용해야 함).
    일부 이미지를 수정해보기(예를 들면 회전, 뒤집기, 어둡게 하기).
    모델이 이상치를 감지하는지 확인하기(즉 정상 샘플과 이상치에 대해 score_samples() 메서드 출력을 비교)

13. 일부 차원 축소 기법은 이상치 탐지를 위해서 사용할 수도 있음. 예를 들어 올리베티 얼굴 데이터셋을 PCA를 사용해 분산의 99% 유지하도록 축소해보고, 그다음 각 이미지의 재구성 오차를 계산.
    다음으로 이전 연습문제에서 만든 수정된 이미지를 선택해 재구성 오차를 확인하기.
    재구성 오차가 얼마나 커지는지 확인하고, 재구성 이미지를 출력해보면 이유를 알 수 있음.
    정상 얼굴을 재구성하기 때문임.